

Attention ② 작은 Transformer 만들기

multi-head attention과 pre-norm block을 구현한다.

소형 decoder 언어 모델을 학습하고 sampling을 비교한다.

데이터 누출·인과성·평가 조건을 검사한다.

BLOCKS
→ **LANGUAGE**

00-02분 · 이번 주에 만들 결과물 확인

90분 수업 지도

- 01 00-05 도입 · 05-20 개념 · 20-35 수식과 손계산
- 02 35-50 구현 설계 · 50-75 실습 · 75-85 분석
- 03 85-90 회고 · 부록과 자율 과제는 수업 이후

90 MIN / 개념 45 + 실습 25 + 분석·회고 20

02-05분 · 실습 노트북 열기

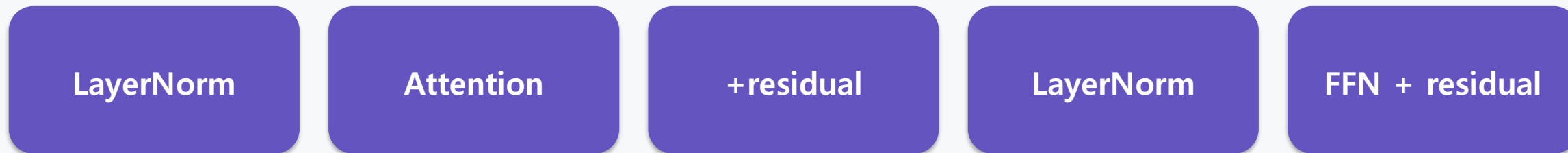
단일 헤드에서 multi-head로



- 01 같은 입력을 여러 부분 공간으로 투영한다.
- 02 각 헤드가 별도의 attention 분포를 계산한다.
- 03 헤드 출력을 합치고 출력 투영을 적용한다.

05-20분

블록의 두 하위 연산



- 01 attention은 토큰 사이 정보를 섞는다.
- 02 FFN은 각 위치의 채널 표현을 비선형으로 바꾼다.
- 03 각 하위 연산에 residual 경로를 둔다.

05-20분

LayerNorm과 BatchNorm

- 01 LayerNorm은 보통 각 토큰의 마지막 특징 축을 정규화한다.
- 02 배치의 다른 문장 통계를 모으지 않는다.
- 03 train/eval에서 BatchNorm처럼 running 통계를 전환하지 않는다.

LayerNorm(D): 각 [D] 벡터를 정규화

05-20분

pre-norm으로 쌓기

- 01 하위 연산 전에 LayerNorm을 적용한다.
- 02 residual 경로는 정규화 바깥으로 통과한다.
- 03 최종 블록 뒤에도 LayerNorm을 적용한다.

$$\mathbf{x} \leftarrow \mathbf{x} + \text{Attn}(\text{LN}(\mathbf{x})); \mathbf{x} \leftarrow \mathbf{x} + \text{FFN}(\text{LN}(\mathbf{x}))$$

05-20분

학습과 생성의 계산 차이

- 01 학습은 정답 문맥으로 모든 위치를 병렬 계산한다.
- 02 생성은 새 토큰 하나를 뽑은 뒤 다시 실행한다.
- 03 이전 예측의 실수가 다음 문맥에 들어간다.

teacher forcing / autoregressive generation

05-20분

decoder-only 모델의 범위

- 01 token·position embedding과 여러 block을 연결한다.
- 02 마지막 Linear가 다음 문자 logits를 만든다.
- 03 지시 따르거나 대화 정렬 학습은 포함하지 않는다.

작은 next-token 모델 ≠ 완성된 대화 서비스

05-20분

좋은 평가 단위

- 01 같은 held-out 토큰에 CE와 perplexity를 계산한다.
- 02 최고 validation checkpoint를 선택한다.
- 03 test는 선택 완료 후 한 번 사용한다.

토큰 수로 가중 평균한 CE $\rightarrow \exp(\text{CE})$

05-20분

헤드 shape 변환

- 01 $d_{\text{model}}=64$, $\text{heads}=4$ 이면 $d_{\text{head}}=16$ 이다.
- 02 $[B, T, 64]$ 를 $[B, T, 4, 16]$ 으로 나눈다.
- 03 transpose 후 $[B, 4, T, 16]$ 으로 attention한다.

$$[B, H, T, d] @ [B, H, d, T] \rightarrow [B, H, T, T]$$

20-35분

헤드 결합하기

결과 $[B, H, T, d]$ 를 $[B, T, H, d]$ 로 돌린다.

reshape로 $[B, T, D]$ 를 만든다.

W_O 로 헤드 정보를 섞는다.

```
y = y.transpose(1, 2)
y = y.contiguous().view(B, T, D)
y = self.out(y)
```

20-35분

FFN의 폭과 파라미터

- 01 $D \rightarrow 4D \rightarrow D$ 의 두 선형층을 사용한다.
- 02 중간에 GELU와 dropout을 넣을 수 있다.
- 03 attention 외에 FFN도 큰 파라미터 비중을 가진다.

FFN weight count $\approx 8D^2$ (bias 제외)

20-35분

LayerNorm 계산

- 01 한 토큰의 D개 값에서 평균과 분산을 구한다.
- 02 표준화 후 학습 가능한 affine 변환을 한다.
- 03 배치 크기 B에 직접 의존하지 않는다.

$$\text{LN}(h) = \gamma \odot (h - \text{mean}(h)) / \sqrt{\text{var}(h) + \epsilon} + \beta$$

20-35분

top-k sampling

- 01 vocabulary 중 점수가 높은 k개만 남긴다.
- 02 나머지는 $-\infty$ 로 바꾼 뒤 softmax한다.
- 03 $k=1$ 이면 최대 점수 토큰만 선택한다.

$$\text{logits} < \text{kth_value} \rightarrow -\infty$$

20-35분

perplexity 해석

- 01 CE가 $\ln(10)$ 이면 perplexity는 10이다.
- 02 확률적으로 유효한 선택지 수라는 직관으로 읽을 수 있다.
- 03 정확한 후보 개수를 세는 지표는 아니다.

$$\text{PPL} = \exp(\text{평균 token NLL})$$

20-35분

모듈을 네 부분으로 나누기

Embedding

Block × 2

LayerNorm

LM head

- 01 CausalMHA는 헤드 분할·mask·결합을 맡는다.
- 02 Block은 LN·residual·FFN을 묶는다.
- 03 TinyTransformer는 embedding과 block stack·head를 연결한다.

35-50분

dropout이 평가를 흔들지 않게

- 01 학습에서는 `model.train()`으로 dropout을 활성화한다.
- 02 평가·생성·causal 검사에서는 `model.eval()`을 사용한다.
- 03 평가를 마친 후 학습 모드를 복원한다.

반복 forward가 같은지 eval에서 확인

35-50분

학습 안정성 점검

- 01 loss가 유한한지 확인하고 gradient norm을 기록한다.
- 02 필요하면 clip_grad_norm_으로 큰 gradient를 제한한다.
- 03 optimizer는 모든 학습 파라미터를 받아야 한다.

zero_grad → backward → clip → step

35-50분

실험 예산을 명시하기

- 01 steps·batch·context 길이로 처리 토큰 수를 계산한다.
- 02 비교할 때 parameter 수와 학습 시간을 함께 기록한다.
- 03 하나의 seed 결과는 탐색 결과로 표현한다.

$$\text{processed tokens} = \text{steps} \times B \times T$$

35-50분

checkpoint에 넣을 것

- 01 model state와 구조 설정을 저장한다.
- 02 vocabulary와 토큰화 규칙도 함께 저장한다.
- 03 생성을 재현하려면 prompt-sampling 설정·seed를 남긴다.

weights + config + vocabulary + run settings

35-50분

LAB 1 · multi-head 구현 검사

- 01 practice §1–3: head shape와 causal mask를 확인한다.
- 02 미래 토큰 변경에 과거 logits가 불변인지 검사한다.
- 03 모델 파라미터 수를 출력한다.

50–59분 / 산출물: shape·causality assert

50–75분 · practice.ipynb와 함께 진행

LAB 2 · 학습과 평가

- 01 practice §4: TinyTransformer를 짧게 학습한다.
- 02 train/validation CE 곡선을 저장한다.
- 03 최고 val state를 복원하고 test CE를 구한다.

59–69분 / 산출물: CE 곡선 + 최종 PPL

50–75분 · practice.ipynb와 함께 진행

LAB 3 · 생성 제어

- 01 practice §5–6: greedy, temperature, top-k를 비교한다.
- 02 동일 prompt와 seed를 사용하고 반복성을 관찰한다.
- 03 checkpoint를 저장·복원해 logits가 같은지 확인한다.

69–75분 / 산출물: 4개 생성 결과 + 복원 검사

50–75분 · practice.ipynb와 함께 진행

어떤 샘플링이 좋은가

- 01 greedy는 반복되고 단조로울 수 있다.
- 02 온도를 높이면 낮은 확률 문자도 더 자주 나온다.
- 03 top-k는 후보를 제한하지만 사실성을 보장하지 않는다.

유창성 / 반복 / 다양성 / 목적

75-85분

헤드 수 실험 해석

- 01 D를 고정하면 head당 차원이 달라진다.
- 02 head 수가 많아도 성능이 항상 좋아지지 않는다.
- 03 같은 예산과 여러 seed로 비교한다.

$$H \uparrow \Rightarrow d_{\text{head}} = D/H \downarrow$$

75-85분

학습 데이터와 생성 결과

- 01 훈련 문장의 반복을 생성 능력으로 과장하지 않는다.
- 02 새 조합에서도 규칙을 유지하는지 관찰한다.
- 03 합성 코퍼스 성능을 일반 자연어로 외삽하지 않는다.

암기 / 조합 일반화 / 실제 언어 능력 구분

75-85분

출구 퀴즈

- 01 Q1. $D=96$, $H=6$ 이면 head 차원은?
- 02 Q2. FFN은 서로 다른 위치를 직접 섞을까?
- 03 Q3. 생성 시 dropout을 끄는 호출은?

shape / 연산 역할 / 실행 모드

85-90분

다음: 확률 분포에서 이미지 생성

- 01 언어 모델은 다음 토큰의 분포를 학습했다.
- 02 VAE는 잠재변수와 이미지의 확률적 관계를 학습한다.
- 03 우도·샘플링·표현이라는 공통 주제로 이어진다.

다음: z 를 뽑아 새로운 이미지를 만든다

85-90분

부록 · sinusoidal position

- 01 위치마다 사인·코사인 주파수를 조합한다.
- 02 학습형 위치 embedding과 다른 설계 선택이다.
- 03 더 긴 길이의 성능이 자동 보장되는 것은 아니다.

$$PE(pos, 2i) = \sin(pos / 10000^{(2i/D)})$$

수업 외 심화

부록 · KV cache

- 01 생성 중 과거 key와 value를 저장해 재사용한다.
- 02 새 토큰의 query 중심으로 추가 계산한다.
- 03 cache 길이와 위치 인덱스 관리가 필요하다.

반복 계산 감소 ↔ cache 메모리 증가

수업 외 심화

부록 · weight tying

- 01 token embedding과 출력 projection 가중치를 공유할 수 있다.
- 02 vocabulary 관련 파라미터 수를 줄인다.
- 03 shape와 초기화 전략을 확인한다.

```
lm_head.weight = token_embedding.weight
```

수업 외 심화

부록 · top-p sampling

- 01 정렬된 확률의 누적합이 p 에 닿는 후보 집합을 쓴다.
- 02 확신 수준에 따라 후보 수가 달라진다.
- 03 최소 한 개의 후보를 유지해야 한다.

고정 k / 가변 nucleus

수업 외 심화

부록 · SDPA 사용 시 주의

- 01 fused attention API가 속도·메모리를 개선할 수 있다.
- 02 dropout_p는 eval에서 자동 0이 된다고 가정하지 않는다.
- 03 직접 구현과 dropout=0에서 수치 비교한다.

dropout_p = p if training else 0.0

수업 외 심화

부록 · 긴 문맥과 학습 예산

- 01 T 증가로 한 step의 토큰 수와 attention 비용이 늘어난다.
- 02 같은 steps 비교는 처리 토큰 예산이 달라진다.
- 03 동일 토큰 예산과 동일 시간 예산을 구분한다.

공정성의 기준을 먼저 정한다

수업 외 심화

부록 · 작은 프로젝트 제안

- 01 head 수 또는 FFN 폭을 한 가지 바꾼다.
- 02 학습 곡선과 생성 오류 유형을 함께 비교한다.
- 03 세 번 반복하고 평균·변동을 보고한다.

자율 과제: top-k + block + head ablation

수업 외 심화

시각 보충 · 미래를 가리는 삼각형

1	0	0	0	0
1	1	0	0	0
1	1	1	0	0
1	1	1	1	0
1	1	1	1	1

행 = query / 열 = key

1 = 허용 / 0 = 차단 (논리 mask)

softmax 후 각 행의 확률 합 = 1

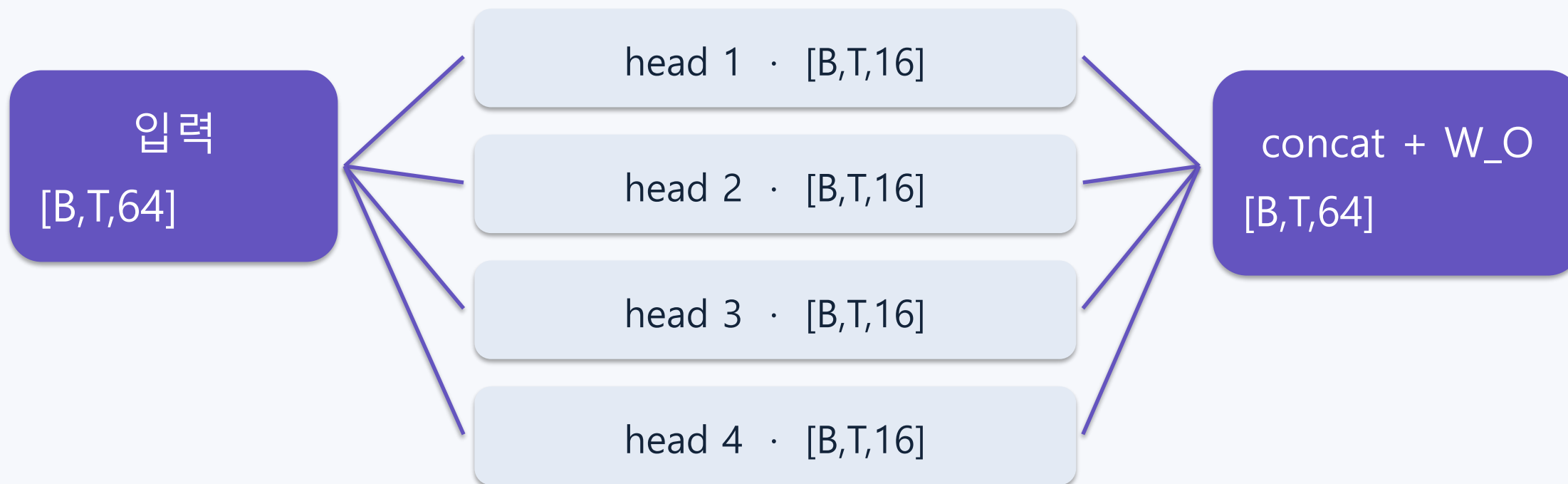
위치 2는 0·1·2만 참조한다.

입력의 현재 토큰은 허용하고

그 다음 토큰을 정답으로 예측한다.

수업 외 복습 · 해당 개념 설명 시 함께 활용 가능

시각 보충 · head 분할과 결합

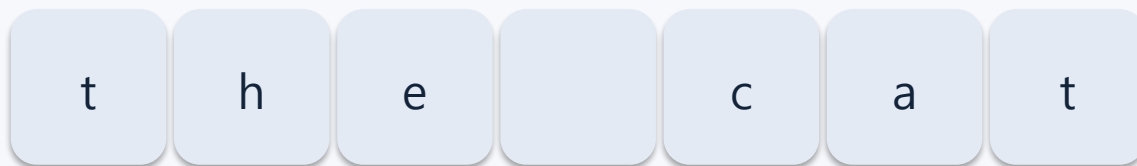


head 축을 복원할 때 $\text{transpose} \rightarrow \text{contiguous} \rightarrow \text{view}$ 순서를 확인한다.

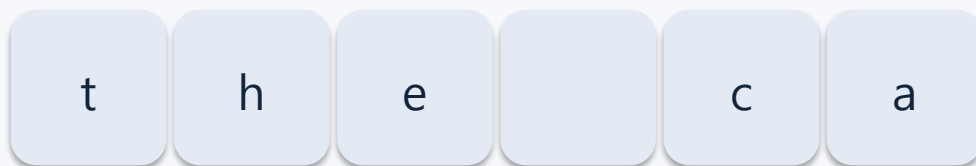
수업 외 복습 · 해당 개념 설명 시 함께 활용 가능

시각 보충 · next-token shift

원본



입력 x



정답 y



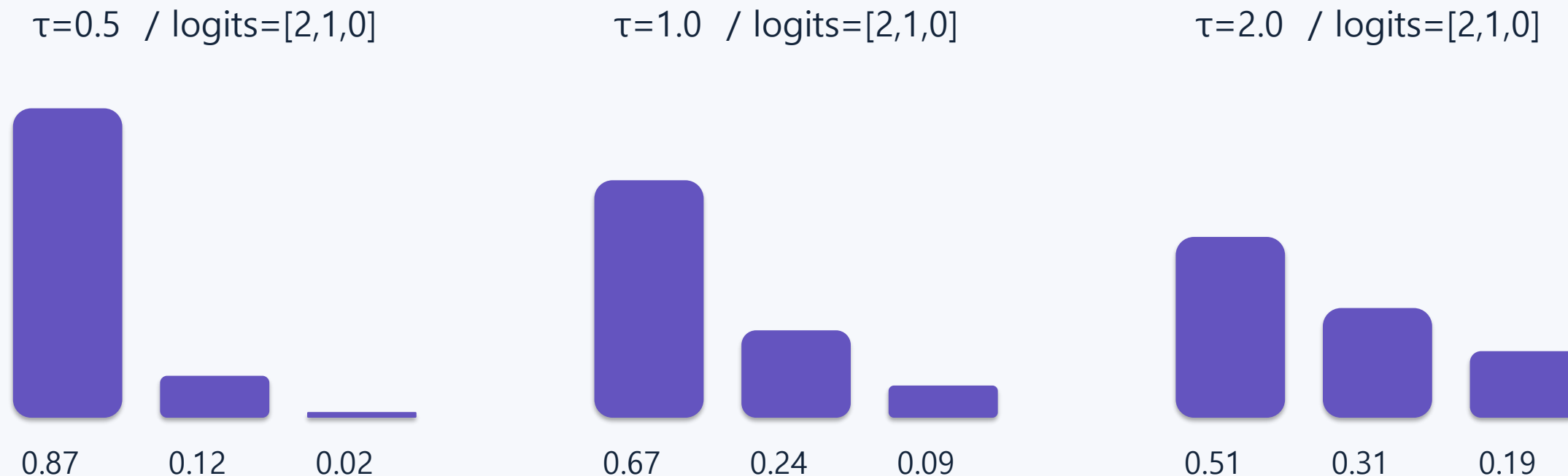
같은 열에서

$x_t \rightarrow y_t$

정답은 원본의 다음 위치

수업 외 복습 · 해당 개념 설명 시 함께 활용 가능

시각 보충 · temperature와 확률



설명용 확률 계산. 온도는 모델 가중치를 바꾸지 않고 선택 분포를 바꾼다.

수업 외 복습 · 해당 개념 설명 시 함께 활용 가능

읽을거리와 출처

- 01 Attention Is All You Need · Vaswani et al.
- 02 Scaled Dot Product Attention · 공식 API
- 03 CrossEntropyLoss · 공식 API

원문 링크는 발표자 노트와 주차별 README에 수록

생각하기 → 손으로 계산하기 → 코드로 검증하기